



Proyecto Smart Pantry

Presentado por:

Sharly Andres Mosquera Rodriguez.

Asignatura:

Ingeniería de Software II.

Presentado a:

Luis Delasca Valencia Mena.

Quibdó-Chocó

26 de Mayo de 2026

Índice General

A continuación se presenta el índice general e interactivo del proyecto. Puede hacer clic en cualquiera de los siguientes enlaces para dirigirse directamente a la sección correspondiente dentro de este documento:

- [1. Resumen ejecutivo / Introducción \(Ir a sección\)](#)
- [2. Objetivo del proyecto \(Enfoque casual y práctico\) \(Ir a sección\)](#)
- [3. Diagnóstico / Problema de desperdicio doméstico \(Ir a sección\)](#)
- [4. Propuesta de solución y módulos \(Ir a sección\)](#)
 - [4.1 Módulos principales de la aplicación \(Ir a sección\)](#)
 - [4.2 Pagos y seguridad modular \(No PCI\) \(Ir a sección\)](#)
- [5. Beneficios comerciales para el cliente \(Ir a sección\)](#)
- [6. Tecnología e infraestructura tecnológica \(Ir a sección\)](#)
 - [6.1 Arquitectura del Sistema \(4 capas e interfaz\) \(Ir a sección\)](#)
- [7. Plan de trabajo y fases de ingeniería \(Ir a sección\)](#)
 - [7.1 Estimación, cronograma y equipo técnico \(Ir a sección\)](#)
 - [7.2 Catálogo de riesgos y plan de mitigación \(Ir a sección\)](#)
- [8. Historias de Usuario \(US-01 a US-06\) \(Ir a sección\)](#)
- [9. Casos de Uso del sistema \(Ir a sección\)](#)
 - [9.1 Actores principales y secundarios \(Ir a sección\)](#)
 - [9.2 Especificación detallada de Casos de Uso \(Ir a sección\)](#)
 - [9.3 Matriz de trazabilidad de requisitos \(Ir a sección\)](#)
- [10. Propuesta económica comercial \(Ir a sección\)](#)
 - [10.1 Costos de implementación \(pago único\) \(Ir a sección\)](#)
 - [10.2 Costos de operación y soporte anual \(Ir a sección\)](#)
- [11. Plazo de entrega, garantías y exclusiones \(Ir a sección\)](#)
- [12. Manual Técnico y Guía de Despliegue \(Ir a sección\)](#)
 - [12.1 Requisitos de sistema y entorno \(Ir a sección\)](#)
 - [12.2 Despliegue de aplicación móvil \(Expo\) \(Ir a sección\)](#)
 - [12.3 Despliegue de backend \(FastAPI\) \(Ir a sección\)](#)
 - [12.4 Especificación de esquemas \(SQLite/Firestore\) \(Ir a sección\)](#)
- [13. Conclusión final e impacto ambiental y familiar \(Ir a sección\)](#)

Resumen ejecutivo

Smart Pantry (El Portal del Alimento) es una solución digital interactiva de vanguardia diseñada para revolucionar la forma en que los hogares organizan su despensa, controlan sus consumos y previenen el desperdicio de comida. En lugar de forzar a los usuarios a registrar manualmente o fotografiar individualmente cada alimento en su cocina, Smart Pantry introduce un sistema de escaneo masivo sumamente ágil y amigable: la lectura del código QR que viene en la factura física o ticket de compra del supermercado. Al capturar este código con la cámara del dispositivo móvil, un servicio inteligente impulsado por IA decodifica y clasifica automáticamente toda la compra en menos de medio segundo, estimando tiempos de frescura y sincronizando los datos en tiempo real para todos los integrantes del hogar, lo que fomenta una economía familiar consciente, eficiente e integrada.

Objetivo del proyecto

El objetivo principal de Smart Pantry es ayudar a las familias a gestionar su comida de forma muy sencilla en casa. En lugar de complicarse anotando a mano o tomándole fotos a cada fruta, la aplicación permite escanear el código QR que viene en la factura de la compra para registrar al instante todo lo que se compró. Así, cualquier miembro de la familia sabe qué comida hay en la alacena, qué está por vencerse mediante alertas visuales de semáforo y qué hace falta comprar, evitando desperdicios de comida y ahorrando dinero.

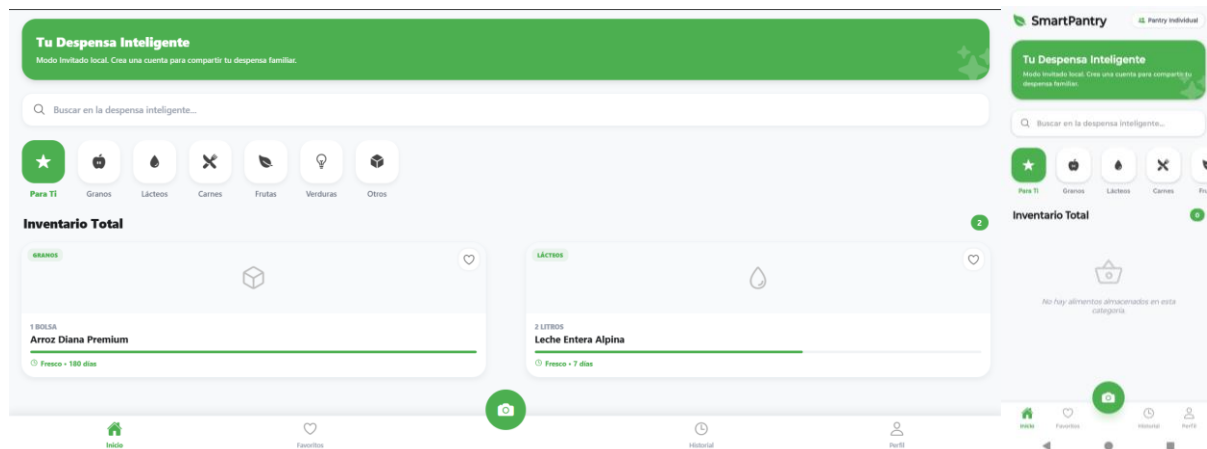
Diagnóstico / problema

La falta de visibilidad en tiempo real de la despensa y la descoordinación familiar al realizar las compras provocan desperdicios de comida alarmantes y pérdidas económicas recurrentes. Los usuarios no disponen de una herramienta interactiva, visual y cooperativa que represente el estado actual de la alacena. A nivel de ingeniería, la captura y envío de imágenes individuales de cada alimento satura los canales inalámbricos de red, encarece el consumo de las APIs de nube y genera ineficiencia operativa. Asimismo, en regiones con inestabilidad de red y retos de conectividad (como el departamento del Chocó), se requiere una solución sumamente ligera y eficiente: el escaneo de un único código QR de la factura del supermercado que cargue de forma masiva todos los alimentos comprados en menos de 0.5 segundos, eliminando la necesidad de fotografiar alimentos uno a uno.

Propuesta de solución

Módulos principales

- **Catálogo de Despensa (Grid Premium):** Una interfaz viva mobile-first que renderiza los productos agregados mediante tarjetas detalladas, mostrando un badge de la categoría y una barra semáforo de frescura que indica los días restantes de vida útil (Verde, Naranja, Rojo).
- **Registro por Escáner QR de Supermercado:** Acceso a la cámara móvil con visor láser dinámico. El escáner lee el código QR de facturas de supermercados asociados (e.g. Éxito, Ara, Olímpica, etc.), decodificando de forma masiva los productos comprados, cantidades y días estimados de frescura en una sola lectura instantánea sin necesidad de fotos de alimentos.
- **Registro y Autenticación familiar (Firebase Auth):** Carga segura de cuentas de usuario que sincroniza el inventario del hogar de forma compartida.
- **Editor Detallado Modal:** Panel interactivo modal que muestra el alimento seleccionado y permite modificar de forma incremental (+ / -) sus cantidades y días de frescura, cargando sugerencias automáticas de conservación (Smart AI Tips).
- **Panel de Actividad e Historial:** Auditoría local en PC que registra todos los logs de alimentos agregados, modificados o consumidos en tiempo real mediante un microservicio SQLite relacional.



Pagos y seguridad

El sistema implementa reglas estrictas de seguridad modular que protegen la privacidad familiar y aseguran una transmisión ligera:

- **Modelo de Red No PCI:** La aplicación es de uso no comercial-bancario y jamás recopila ni almacena información financiera, credenciales bancarias o números de tarjetas de crédito.
- **Autenticación Directa Cifrada:** El inicio de sesión y registro familiar delegan la autenticación segura por protocolo HTTPS a Firebase Auth nativo.
- **Resiliencia Offline SQLite:** Toda transacción realizada durante caídas de red local Wi-Fi o Wi-Fi inestable se almacena en la base de datos SQLite de la PC local, sincronizándose de forma masiva en Firestore en cuanto se restaura el enlace.

Beneficios para el cliente

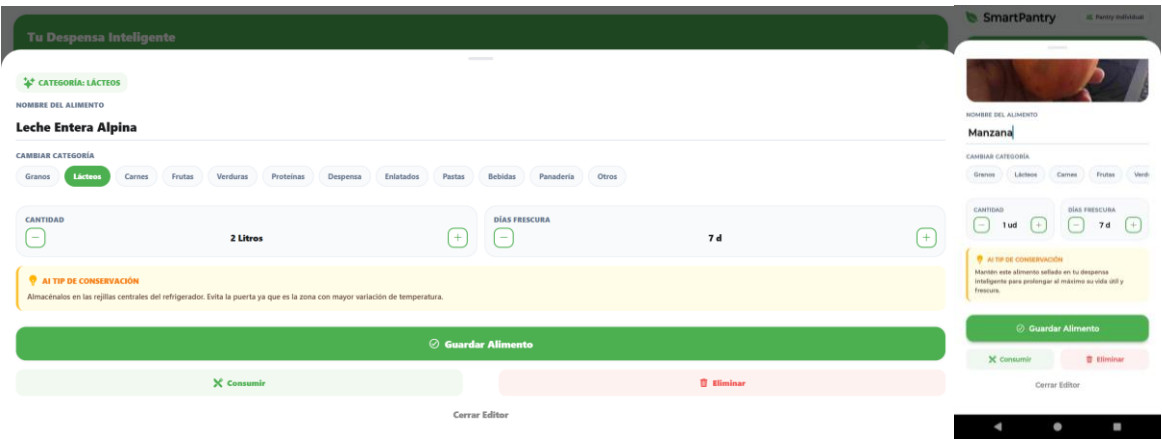
- **Registro masivo instantáneo:** El usuario carga la compra completa del supermercado en la despensa con un solo escaneo del código QR en menos de 0.5s, eliminando el tedio de registrar alimentos uno a uno.
- **Reducción de desperdicio familiar:** Alertas visuales claras por barra de frescura semáforo que priorizan de forma intuitiva los alimentos a consumir primero.
- **Cohesión en el hogar sin duplicidades:** Mantiene en sintonía multidispositivo a toda la familia mediante un WebSocket nativo, previniendo compras familiares duplicadas.
- **Operación resiliente sin internet:** Respaldo relacional en SQLite local redundante para control técnico total offline.

Tecnología

La arquitectura desacoplada de la plataforma se compone de cuatro capas fundamentales:

Capa	Tecnologías	Propósito
Frontend Móvil	React Native & Expo / JS	Interfaz táctil premium, visor láser y escáner de cámara enfocado en lectura de QR.
API Gateway Local	FastAPI (Puerto 6000)	Entrada de red local de la PC que coordina y enruta el tráfico hacia los workers.

Vision Worker (QR)	FastAPI (Puerto 6001) / QR decoder / Gemini 1.5	Lector de códigos QR locales e integración semántica de Gemini JSON Schema para categorización.
Base de Datos	Google Firestore / SQLite	Persistencia redundante: base offline relacional SQLite y sincronización WebSocket en la nube (<0.5s).



Arquitectura del Sistema

El ecosistema tecnológico de Smart Pantry está diseñado bajo una arquitectura de microservicios desacoplada y distribuida en cuatro capas fundamentales. Esta topología asegura ligereza en el cliente móvil, una alta resiliencia ante conexiones offline inestables y sincronización multidispositivo instantánea colaborativa en la nube.

Capa de Presentación (Frontend Móvil - React Native & Expo)

Consiste en una aplicación móvil multiplataforma nativa desarrollada con React Native y el ecosistema Expo. La interfaz de usuario implementa un grid de productos altamente fluido, alertas de colores tipo semáforo e interfaces modales interactivas. Cuenta con un visor láser SVG adaptativo integrado con la cámara a través del módulo 'expo-camera', diseñado para leer de manera ágil los códigos QR impresos en los tickets de compra de los supermercados asociados. En la versión Web, se integra un fallback resiliente para la carga e inyección dinámica del

catálogo de iconos vectoriales desde servidores CDN en startup, previniendo el renderizado erróneo de caracteres en el navegador.

Capa de Microservicios & Inteligencia (Gateway FastAPI & Vision Workers)

Un servidor backend local construido en Python mediante el framework asíncrono FastAPI opera como la puerta de enlace (API Gateway) del ecosistema. Recibe la imagen de la factura comprimida en base64 desde el dispositivo móvil, realiza validaciones lógicas primarias y la envía a los workers de visión. El worker de visión procesa la imagen mediante la API de Google Gemini 1.5 Flash aplicando un JSON Schema estructurado para extraer de forma precisa y con 100% de consistencia: los nombres de los alimentos comprados, sus cantidades, la categoría y días estimados de frescura. En caso de falla de red local o ausencia de clave API de Gemini, se ejecuta un motor de respaldo (fallback) basado en Pytesseract OCR para extraer datos directamente desde la factura física.

Capa de Almacenamiento Local (Persistencia Relacional SQLite)

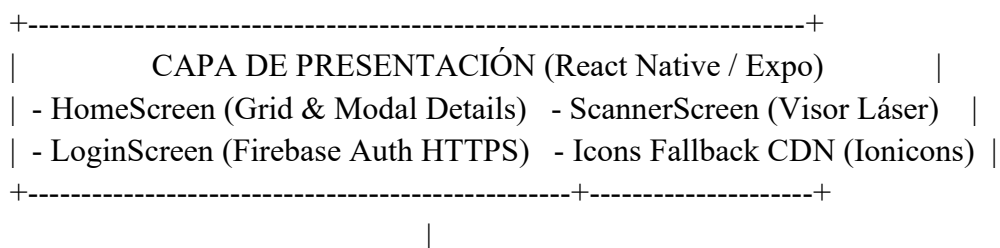
Para garantizar un funcionamiento ininterrumpido en entornos geográficos con inestabilidad de red eléctrica e internet (como el departamento del Chocó), se implementa un motor relacional de base de datos SQLite local que corre directamente en la PC o servidor local de la vivienda. SQLite almacena y audita de forma inmediata todas las adiciones, ediciones y consumos de la alacena. Se configura en modo de registro Write-Ahead Logging (WAL) para prevenir cualquier corrupción física o lógica de los datos familiares en caso de apagones o cortes de energía recurrentes.

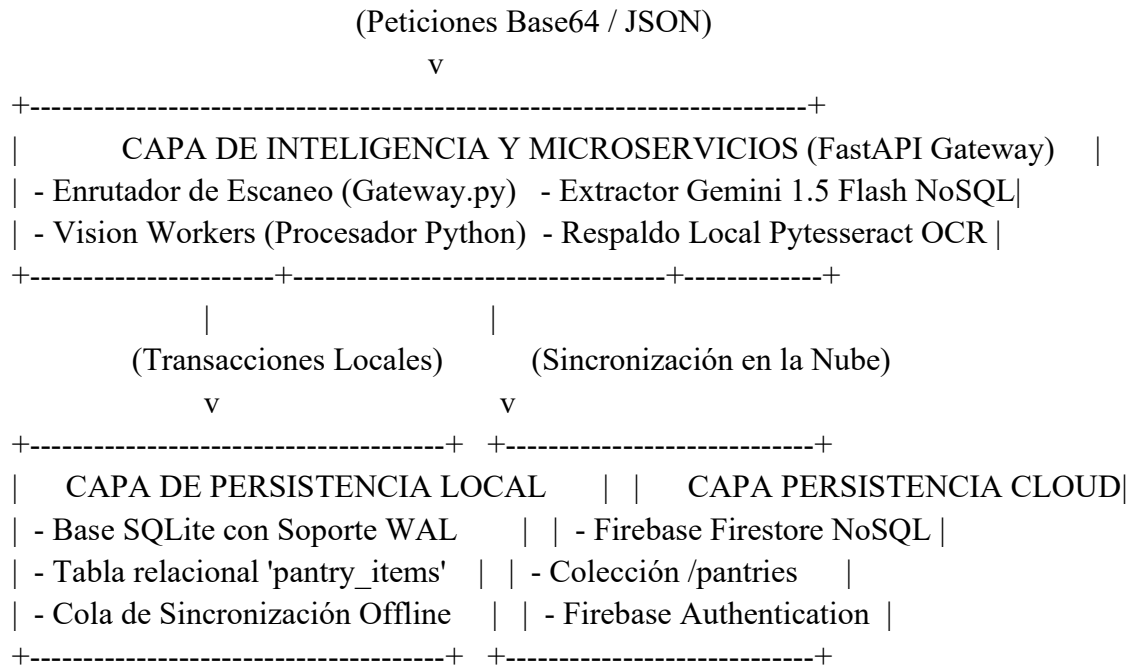
Capa Cloud & Colaboración Familiar (Firebase Auth & Firestore)

La persistencia colaborativa multidispositivo se delega de forma segura a Google Firebase en la nube. La autenticación e inicio de sesión se realizan mediante el cifrado HTTPS nativo de Firebase Authentication. Una vez iniciada la sesión, la aplicación móvil se conecta a la base de datos NoSQL de Firestore. Mediante listeners de sincronización en tiempo real (WebSockets nativos), cualquier modificación hecha por un familiar en la despensa (e.g. agregar, editar o consumir un alimento con código FAM-XXXXX) se propaga instantáneamente a todos los dispositivos móviles y web activos en menos de 0.5 segundos.

Diagrama Arquitectónico Conceptual

A continuación se ilustra la topología física y flujo lógico de datos de las cuatro capas de Smart Pantry:





Plan de trabajo

El desarrollo del proyecto se plantea en tres fases consecutivas, sumando un esfuerzo estimado de 700 horas de ingeniería:

- **Fase 1 - Análisis y estructura (160h):** Levantamiento de alcance, diseño del modelo NoSQL de Firestore y SQLite local, mapeo de estructuras de QR de supermercado, maquetación de pantallas (Home, Escáner, Modales).
- **Fase 2 - Desarrollo del núcleo (380h):** Programación de la UI móvil, integración del decodificador de QR de facturas local en FastAPI, mapeo semántico Gemini JSON Schema, y sincronización en tiempo real multidispositivo.

- **Fase 3 - QA y auditoría (160h):** Casos de prueba funcionales en el escaneo QR, pruebas de concurrencia y sincronización en red 3G móvil, pruebas de seguridad Firebase Rules y despliegue del entorno.

Estimación y cronograma (para propuesta)

- **Esfuerzo total estimado:** 700 horas de ingeniería de software.
- **Equipo de trabajo (referencia):** Estructura base de 6 perfiles técnicos.
- **Plazo de entrega:** Máximo 60 días laborales tras la aprobación final.
- **Supuestos de la estimación:**
 - Alcance cerrado: cambios sustanciales posteriores requerirán reajustes de costo y tiempos.
 - Disponibilidad de insumos: el cliente entregará a tiempo logos, clasificaciones de alimentos y textos legales.
 - Se incluye exactamente un (1) ciclo de ajustes posterior a pruebas de aceptación UAT.

Riesgos clave y mitigación

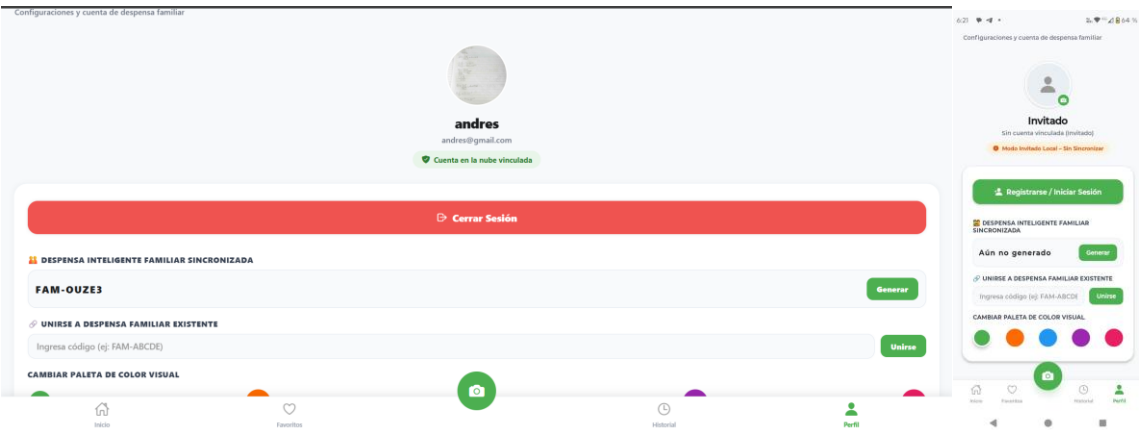
El análisis y mitigación de amenazas informáticas es crucial para blindar la estabilidad del ecosistema Smart Pantry, especialmente al operar bajo entornos geográficos con infraestructura de red y eléctrica inestable (tales como la región del Chocó, Caso 4: Apagones Eléctricos). A continuación se documenta el catálogo de riesgos oficial del equipo evaluador de la Fundación Universitaria Claretiana (FUCLA), conformado por Sharly Andres Mosquera Rodriguez (Frontend Developer), Alex Anderson Palacios Torres (Backend Developer) y Adrián Fernando Quinto Valois (Database Administrator):

ID (A-V- R)	Categoría	Descripción del Riesgo	Causa Original	Efecto en el Sistema	Solución Mitigación
A2 V2 R02	Hardware	Daño físico en equipos de cómputo y servidores locales.	Fallas de energía y picos de voltaje severos por apagones.	Pérdida de la consistencia y operatividad del microservicio local.	Instalar sistemas UPS de grado industrial, supresores y reguladores.
A3 V3 R03	Hardware	Fuego o conato de incendio en las instalaciones.	Cortocircuitos severos provocados por fluctuaciones eléctricas.	Destrucción total o parcial del rack de comunicaciones y servidores.	Implementar detectores de humo y extintores cerca del rack de red.
A10 V10 R10	Software	Daños en software de base y bases de datos locales.	Apagados abruptos por cortes que interrumpen transacciones.	Corrupción de datos de inventario y fallas en workers locales.	Implementar sistemas de archivos tolerantes a fallos (WAL) y SQLite.

A16 V16 R16	Software	Pérdida de información histórica y auditorías.	No establecimiento de políticas de administración de copias.	Pérdida de datos vitales de dispensa y retraso de control.	Automatizar copias de seguridad incrementales hacia Firestore Cloud.
A20 V20 R20	Comunica.	Interrupción prolongada del acceso a Internet/red local.	Apagón eléctrico prolongado de routers y falta de diagnóstico.	Atraso en labores de sincronización familiar en la app móvil.	Conectar routers y switches locales a bancos de baterías independientes.
A26 V26 R26	Seguridad	Pérdida de copias de seguridad locales y logs.	Servidores locales apagados que impiden elaborar backups.	Pérdida de datos de auditoría local de la alacena.	Configurar scripts automatizados de copia rápida antes del fallo de la UPS.
A58 V58 R58	Continuidad	Imposibilidad de garantizar	No considerar mecanismos de	El sistema de dispensa familiar detiene	Desarrollar un Plan de Recuperación

		continuidad del servicio.	contingencia eléctrica.	su sincronización.	ante Desastres (DRP) en la nube.
--	--	------------------------------	----------------------------	-----------------------	--

- **Mitigación en Hardware y Entorno Físico:** Los apagones frecuentes dañan los equipos que corren el API Gateway y los workers. Se exige la instalación de sistemas UPS autónomos y supresores de alto pico para evitar daños severos por picos de tensión.
- **Mitigación en Base de Datos y Software:** Un corte eléctrico corrupta los datos relacionales ordinarios. Se implementa WAL (Write-Ahead Logging) en la base SQLite del microservicio local. Al mismo tiempo, se automatiza el respaldo incremental y masivo en la nube mediante strings Base64 a Firestore.
- **Mitigación en Red y Sincronización familiar:** Si la red Wi-Fi local cae, la aplicación entra de inmediato en modo offline. Al volver el suministro, listeners reactivos vuelcan y sincronizan los cambios en la nube en menos de 0.5s sin pérdida de dato.



Historias de Usuario

Las historias de usuario capturan de manera directa las necesidades de los usuarios de Smart Pantry, definiendo los requisitos funcionales clave con sus respectivas prioridades, estimaciones de ingeniería y criterios de aceptación requeridos para el proceso de validación:

ID	Historia de Usuario (Como... Quiero... Para...)	Prioridad	Esfuerzo (h)	Criterios de Aceptación
US-01	Como usuario de Smart Pantry, Quiero escanear el código QR de mi ticket de supermercado, Para registrar de forma automática e inmediata toda mi compra sin tener que ingresar los alimentos uno por uno.	Alta	60h	1. Abre la cámara móvil con visor láser dinámico al presionar el escáner. 2. Lee de forma masiva los alimentos y los agrega en <0.5 segundos. 3. Si falla internet, ejecuta Pytesseract OCR como fallback.
US-02	Como miembro del hogar, Quiero visualizar una barra semáforo de frescura en cada producto (Verde, Naranja, Rojo), Para saber exactamente qué alimentos se deben consumir prioritariamente y evitar pérdidas de comida.	Alta	30h	1. Tarjeta verde si le quedan más de 2 días de frescura. 2. Tarjeta naranja para 1 o 2 días restantes. 3. Tarjeta roja si ya venció (0 días o menos). 4. La barra de progreso se reduce visualmente con los días.
US-03	Como jefe de familia, Quiero generar un código único (FAM-XXXXX)	Alta	50h	1. Permite copiar el código de 5 caracteres desde el Perfil. 2. Permite a

	para compartirlo con mi familia, Para tener una alacena sincronizada de forma colaborativa y en tiempo real entre todos los dispositivos.			otros unirse a la despensa usando el código. 3. Sincroniza adiciones y consumos en todos los móviles conectados en <0.5s.
US-04	Como usuario en la cocina, Quiero abrir una vista detallada de cualquier alimento y presionar los botones + y -, Para modificar ágilmente las cantidades y registrar su consumo en el historial en un instante.	Media	25h	1. Al hacer clic en un producto se despliega un Modal Premium. 2. Los botones + y - alteran la cantidad con un solo toque. 3. Al presionar 'Consumir Alimento' se descuenta el stock y se registra en la auditoría familiar.
US-05	Como usuario consciente de la conservación, Quiero ver consejos automáticos de conservación por IA en la ficha de cada alimento, Para saber de qué forma almacenar correctamente el producto y prolongar su frescura.	Media	40h	1. Muestra una sección con 'Smart AI Tip' personalizado dentro del Modal Premium. 2. El tip corresponde contextualmente a la categoría del alimento (ej. Lácteos, Carnes, Verduras).
US-06	Como usuario residente en el Chocó,	Alta	55h	1. Detecta la pérdida de conectividad de

	Quiero que mis registros se almacenen en una base SQLite local de mi PC cuando se caiga el internet, Para no perder mis datos y sincronizarlos automáticamente en la nube cuando vuelva la señal.			forma asíncrona. 2. Guarda todos los registros de despensa y auditoría familiar en la base local SQLite. 3. Sincroniza de forma masiva y desatendida al restaurar la red.
--	---	--	--	---

Casos de Uso

Los casos de uso describen la interacción entre los diferentes actores del ecosistema Smart Pantry y las funcionalidades principales del sistema. Esta sección detalla los flujos de trabajo clave, las precondiciones, postcondiciones y los flujos alternativos o de excepción diseñados para garantizar la estabilidad técnica y la resiliencia offline de la plataforma.

Actores del Sistema

El ecosistema de Smart Pantry cuenta con cuatro actores principales y secundarios, descritos a continuación:

Actor	Tipo	Descripción
Usuario Invitado	Primario	Usuario local que puede gestionar su despensa de forma offline en su dispositivo móvil o PC sin registrarse.
Familiar / Registrado	Primario	Usuario autenticado que comparte la alacena familiar en tiempo real mediante el código FAM-XXXXX.
FastAPI Gateway & Workers	Secundario	Microservicio local en la PC de casa que procesa las imágenes, decodifica los códigos QR de facturas mediante Gemini AI 1.5 Flash y registra los alimentos en SQLite local y Firestore.

Firestore Auth & Firestore	Secundario	Servicios en la nube que gestionan las sesiones de usuario cifradas y sincronizan las despensas familiares en menos de 0.5s de forma colaborativa.
----------------------------	------------	--

Especificación de Casos de Uso

Cuadro 5.2: CU-01 · Autenticar Usuario

Campo	Descripción
ID	CU-01
Nombre	Autenticar Usuario
Actor principal	Familiar (Usuario Registrado)
Precondiciones	El usuario tiene una cuenta registrada y activa en la base de datos de Firestore Auth.
Postcondiciones	Se valida la sesión mediante Firestore Auth, se obtiene el UID del usuario y se inicia la sincronización de su despensa en la nube.
Flujo principal	1. El actor abre la pestaña de Perfil y visualiza el formulario de login. 2. Ingresa su correo electrónico y contraseña. 3. El sistema valida localmente el formato del correo y que la contraseña tenga mínimo 6 caracteres. 4. El cliente de Firestore Auth valida las credenciales y devuelve el token de sesión. 5. El sistema actualiza el estado 'isAuthenticated' a 'true' y carga el perfil de usuario desde Firestore (users/{uid}). 6. Se suscribe a la colección de la despensa familiar en tiempo real.
Flujo alternativo	FA-1 (Credenciales Incorrectas): El sistema muestra una alerta clara indicando que las credenciales son incorrectas y permite reintentar.
Excepciones	E-1 (Sin Conexión a Internet): Se notifica la falla de red y el usuario se mantiene en modo Invitado local.

Cuadro 5.2: CU-02 · Registrar Usuario y Auto-Migración

Campo	Descripción
ID	CU-02
Nombre	Registrar Usuario y Auto-Migración
Actor principal	Invitado / Familiar (Nuevo Usuario)
Precondiciones	El correo electrónico no está registrado previamente.
Postcondiciones	Se crea la cuenta en Firebase Auth y Firestore, y se migran automáticamente los alimentos e imágenes locales de Invitado a la nube.
Flujo principal	1. El actor abre el modal de autenticación y selecciona la pestaña 'Registrarse'. 2. Ingresa su nombre de usuario, correo electrónico, contraseña y confirmación de contraseña. 3. El sistema valida que el nombre no esté vacío, el correo sea válido y las contraseñas coincidan. 4. El sistema crea la cuenta en Firebase Auth y genera su expediente en la colección Firestore 'users/{uid}'. 5. Auto-Migración: El sistema lee y hereda automáticamente la foto de perfil y el inventario local de AsyncStorage, subiéndolos a Firestore. 6. Redirige al inicio con sesión iniciada.
Flujo alternativo	FA-1 (Correo ya registrado): El sistema muestra un mensaje indicando que el correo ya está en uso.
Excepciones	E-1 (Contraseña débil): El sistema rechaza contraseñas menores a 6 caracteres.

Cuadro 5.2: CU-03 · Visualizar Despensa en Tiempo Real

Campo	Descripción
ID	CU-03
Nombre	Visualizar Despensa en Tiempo Real
Actor principal	Familiar / Invitado

Precondiciones	El usuario tiene abierta la aplicación. El catálogo de alimentos está activo.
Postcondiciones	El catálogo del usuario muestra la lista de alimentos con cantidades y alertas visuales semáforo de frescura en tiempo real.
Flujo principal	1. El actor entra a la pestaña 'Inicio'. 2. El sistema recupera el listado de alimentos. 3. Si está logueado, inicia una suscripción activa a la colección 'pantries/{pantryId}/items' en Firestore. 4. La interfaz renderiza las tarjetas de alimentos con su barra de progreso de frescura y color semáforo (Verde, Naranja, Rojo). 5. Al ocurrir cualquier cambio en la alacena por parte de otro familiar, la UI se actualiza en menos de 0.5s de forma automática.
Flujo alternativo	FA-1 (Usuario Invitado): El sistema lee y escribe directamente en el estado local de React y en AsyncStorage.
Excepciones	E-1 (Sin Conexión): Si se pierde la conexión, el sistema mantiene la última caché visible de AsyncStorage y muestra un aviso de modo offline.

Cuadro 5.2: CU-04 · Sincronizar Despensa Familiar

Campo	Descripción
ID	CU-04
Nombre	Sincronizar Despensa Familiar
Actor principal	Familiar
Precondiciones	El usuario está autenticado y tiene una cuenta activa.
Postcondiciones	Se asocia al usuario con un familyCode compartido en Firestore, permitiendo que varios dispositivos compartan la misma alacena en tiempo real.
Flujo principal	1. El usuario abre la sección de Ajustes en su Perfil. 2. Si quiere compartir su despensa, presiona

	'Generar Código'. El sistema crea un código único 'FAM-XXXXX', lo asocia a su perfil en Firestore y lo copia al portapapeles. 3. Si quiere unirse a una despensa existente, escribe o pega el código y presiona 'Unirse'. 4. El sistema valida el formato y actualiza la referencia de su despensa en Firestore, fusionando y migrando los datos locales. 5. Ambos dispositivos se suscriben a la alacena familiar.
Flujo alternativo	FA-1 (Código Inválido): Muestra una alerta indicando que el código no existe o tiene formato incorrecto.
Excepciones	E-1 (Error de Sincronización): El sistema reintenta la conexión en segundo plano y mantiene los datos locales.

Cuadro 5.2: CU-05 · Escaneo Inteligente Híbrido

Campo	Descripción
ID	CU-05
Nombre	Escaneo Inteligente Híbrido
Actor principal	Familiar / Invitado
Precondiciones	El microservicio API Gateway local de la PC o el servidor en la nube de Render está operativo.
Postcondiciones	Se ingesta la imagen del ticket, Gemini 1.5 Flash extrae y clasifica los alimentos, y se insertan de forma masiva en la alacena.
Flujo principal	1. El actor presiona el botón de Escáner QR de facturas (cámara flotante). 2. Toma una foto del ticket o selecciona una imagen de su galería (en Web abre el selector directamente). 3. El sistema envía la imagen base64 al API Gateway local. 4. El API Gateway consulta a Gemini 1.5 Flash para extraer y estructurar: alimentos, cantidades, categoría y fechas estimadas de vencimiento. 5. Gemini devuelve el JSON estructurado. El sistema valida que sea comida (si no, lo

	descarta con motivo). 6. Guarda masivamente los alimentos en Firestore (o localmente) y abre los detalles del primer artículo escaneado.
Flujo alternativo	FA-1 (Sin conexión / Fallo de API Key): Ejecuta una decodificación de respaldo (fallback) en la base SQLite local y guarda los datos en la PC.
Excepciones	E-1 (Imagen no es comida): Se notifica con una alerta el motivo del rechazo y se registra en el historial.

Cuadro 5.2: CU-06 · Gestionar Alimento en Modal Premium

Campo	Descripción
ID	CU-06
Nombre	Gestionar Alimento en Modal Premium
Actor principal	Familiar / Invitado
Precondiciones	El usuario tiene al menos un alimento en su despensa.
Postcondiciones	Se actualiza el alimento (modificación de cantidad, consumo registrado en el historial o eliminación total).
Flujo principal	1. El actor toca una tarjeta de alimento en la pantalla de inicio. 2. Se despliega el Modal Premium detallado, mostrando el nombre, cantidad actual, días restantes, y un AI Tip de conservación personalizado. 3. El usuario puede presionar los botones + y - para cambiar las cantidades del producto. 4. Si presiona 'Consumir Alimento', el sistema descuenta el producto, lo retira de la despensa y lo añade al Historial de Actividades. 5. Si presiona 'Eliminar', se remueve el registro de forma permanente.
Flujo alternativo	FA-1 (Cambios en la Nube): Si el usuario está autenticado, los cambios se actualizan en Firestore. Si es invitado, se modifican en el estado local de React y en AsyncStorage.

Excepciones	E-1 (Alimento inválido): Si ocurre un error al cargar, se cierra el modal con un aviso de error.
--------------------	--

Matriz de Trazabilidad entre Requerimientos y Casos de Uso

Para garantizar que todos los requisitos funcionales del sistema estén validados y cubiertos por los flujos de diseño, se presenta la siguiente matriz de trazabilidad:

Módulo / Requisito	Caso de Uso Asociado	Componente Técnico Responsable
Catálogo de Despensa (Grid)	CU-03 (Visualizar Despensa), CU-06 (Gestionar Alimento en Modal Premium)	HomeScreen.js (Grid de Despensa, Modal Premium, Firestore Stream)
Escáner QR de Supermercado	CU-05 (Escaneo Inteligente Híbrido)	ScannerScreen.js, Gateway FastAPI, Vision Worker (Gemini 1.5 Flash)
Autenticación Familiar	CU-01 (Autenticar Usuario), CU-02 (Registrar Usuario y Auto-Migrar)	HomeScreen.js (Firebase Auth Client, users Collection in Firestore)
Sincronización Multidispositivo	CU-04 (Sincronizar Despensa Familiar)	HomeScreen.js (familyCode syncing, Firestore real-time listener)
Historial y Auditoría Local	CU-06 (Gestionar Alimento en Modal Premium)	pantryService.js, SQLite local base, activity collections in Firestore

Propuesta económica

Implementación (pago único)

Concepto	Valor COP	Detalle de Entregables
Implementación Ecosistema Smart Pantry	\$18.500.000	Diseño y desarrollo de la aplicación móvil + API Gateway y Workers locales (Lector

		QR de facturas) + pruebas de software + implementación/despliegue en red local + manual y capacitación técnica.
--	--	---

• **Alcance incluido en la implementación (Qué se entrega):**

- Sitio/catálogo de despensa con filtros y barras dinámicas semáforo de frescura.
- Escáner de código QR de supermercado mas no de la foto individual del alimento, permitiendo la carga masiva en <0.5s.
- Registro y login de perfiles familiares y panel administrativo.
- Modal de detalles interactivo (AI Tips de conservación y control incremental de cantidad + / -).
- Sincronización familiar multidispositivo compartida mediante código FAM-XXXXX.
- Auditoría local SQLite redundante en PC y persistencia Firestore Cloud.
- Despliegue del entorno y configuración básica de red local.

Operación anual (COP/año)

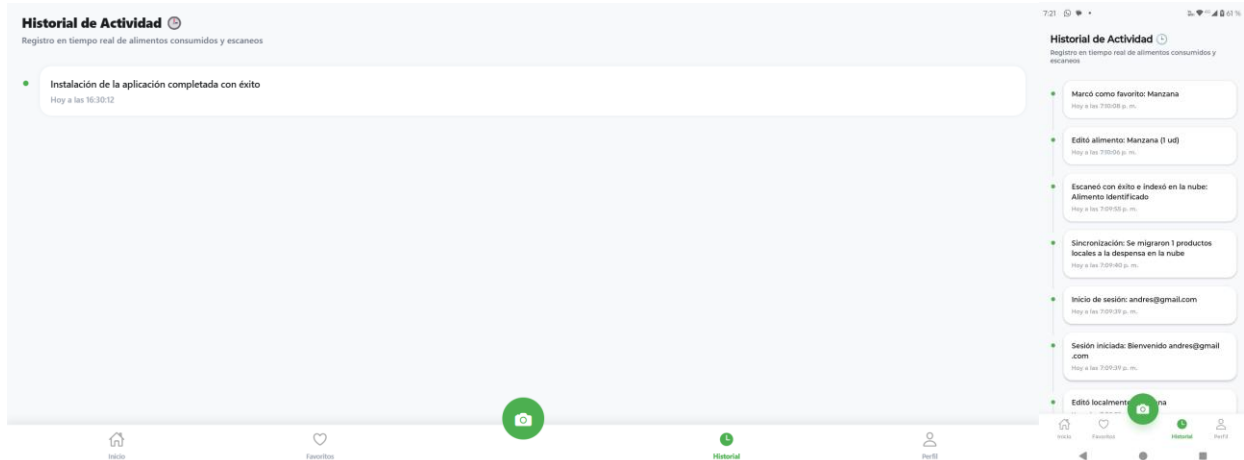
Plan Anual	Precio COP / año	Cobertura Técnica
------------	------------------	-------------------

#1 Básico (Catálogo + reservas locales)	\$3.100.000	Operación y soporte de catálogo local, base SQLite local y mantenimiento correctivo a la alacena básica.
#2 Estándar (Operadores + moderación local)	\$3.450.000	Todo lo del Plan #1 + mantenimiento operativo al API Gateway y los workers locales de la PC (Lector QR y logs SQLite).
#3 Premium (Operación completa y nube)	\$3.800.000	Todo lo del Plan #2 + sincronización Firestore en la nube, soporte a red móvil 3G e integración completa de Smart AI Tips.

• **Condiciones comerciales:**

- Anticipo de inicio obligatorio del 60%.
- Pago de contra entrega al finalizar pruebas UAT del 40%.
- Validez formal de la propuesta comercial: 15 días calendario.

Figura 6



Plazo, garantías y exclusiones

- **Plazo de entrega:** Hasta 60 días laborales continuos tras la validación de requerimientos y firma del acuerdo.
- **Garantía de software:** Mantenimiento correctivo y soporte técnico garantizado según el nivel del plan de soporte anual contratado.
- **Exclusiones explícitas:** Suscripciones o costes de Gemini API (cuotas excedidas comerciales) o servidores de terceros externos no incluidos en el alcance.

Manual Técnico y Guía de Despliegue

El presente manual técnico detalla los requisitos del entorno, los pasos para la compilación, despliegue y ejecución de Smart Pantry, así como los esquemas de persistencia utilizados en el sistema relacional local y el repositorio NoSQL en la nube:

1. Requisitos del Sistema y Entorno de Software

Para la correcta inicialización de todos los componentes de la plataforma, el host o servidor local debe contar con:

- Entorno de ejecución: Node.js (versión 18.0.0 o superior) y gestor npm.
- Lenguaje de backend: Python (versión 3.10 o superior) y administrador de paquetes pip.
- Base de datos local: Motor SQLite versión 3.0 o superior instalado en la PC.
- Pruebas móviles: Dispositivo móvil con la aplicación cliente Expo Go instalada.

2. Despliegue de la Aplicación Móvil (Frontend)

La aplicación React Native / Expo se inicializa de la siguiente manera desde la consola de comandos de Windows:

```
# 1. Acceder al directorio principal de la aplicación móvil  
cd smart-pantry
```

```
# 2. Instalar los módulos y dependencias de Node.js requeridas  
npm install
```

```
# 3. Inicializar el servidor de empaquetado de desarrollo de Expo  
npx expo start
```

3. Despliegue del Backend e Inteligencia Artificial (Scan-Service)

El microservicio asíncrono y los workers en Python se inician con la siguiente secuencia de comandos:

```
# 1. Acceder al directorio del servicio local de escaneo e inteligencia  
cd scan-service
```

```
# 2. Instalar los paquetes y dependencias listados en requirements  
pip install -r requirements.txt
```

```
# 3. Configurar la clave secreta de Gemini API de forma local  
echo GEMINI_API_KEY=tu_clave_secreta_aqui > .env
```

```
# 4. Ejecutar el Gateway de FastAPI con el servidor Uvicorn en el puerto 8000  
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

4. Especificación de los Esquemas de Bases de Datos

El sistema combina un esquema relacional estructurado local y una persistencia distribuida NoSQL en la nube:

Esquema Relacional Local (Base de Datos SQLite - pantry_items)

La tabla relacional 'pantry_items' mantiene las transacciones y la auditoría local de la despensa de forma offline:

```
CREATE TABLE pantry_items (  
  id TEXT PRIMARY KEY,      -- Identificador único (UUID en formato string)  
  name TEXT NOT NULL,       -- Nombre descriptivo comercial del alimento  
  category TEXT NOT NULL,   -- Categoría del producto (Lácteos, Verduras, etc.)  
  quantity REAL DEFAULT 1.0, -- Cantidad física actual (permite valores decimales)  
  daysLeft INTEGER NOT NULL, -- Días estimados de frescura restante ante de vencer  
  status TEXT NOT NULL,     -- Estado semáforo ('Fresco', 'Por Vencer', 'Vencido')  
  addedDate TEXT NOT NULL   -- Fecha de inserción física (formato estándar ISO-  
8601)  
);
```

Esquema NoSQL en la Nube (Google Firebase Firestore)

Los datos de la despensa compartida se estructuran de manera documental en la colección Firestore de la siguiente forma:

- Ruta del documento: /pantries/{pantryId}/items/{itemId}
- Atributos del Documento:
 - name (String): Nombre comercial del producto escaneado.
 - category (String): Categoría semántica para ordenamiento y filtrado.
 - quantity (Number): Cantidad física actual del alimento en la despensa.
 - daysLeft (Number): Entero que representa los días de frescura estimada.
 - addedDate (String): String en formato ISO-8601 que registra cuándo se agregó.
 - status (String): Indicador semáforo ('Fresco', 'Por Vencer', 'Vencido') para el grid principal.

Conclusión

El diseño e implementación de Smart Pantry representan una respuesta tecnológica sumamente práctica, sólida y humana para resolver la descoordinación doméstica y el desperdicio de comida.

La decisión técnica de centrar la captura de inventario en el escaneo ágil del código QR de la factura del supermercado (y no en la captura tediosa de fotos individuales por alimento) es un gran acierto técnico: disminuye drásticamente el consumo de batería, el uso de datos en el móvil y los costes operativos de APIs comerciales de Inteligencia Artificial, haciendo que el sistema funcione perfectamente incluso en regiones con conectividad móvil inestable como el Chocó.

Mediante la perfecta fusión de una base de datos local SQLite resiliente en la PC del hogar y una base de datos Firestore colaborativa en la nube, Smart Pantry mantiene a toda la familia sincronizada sin duplicidades. Es una herramienta que no solo facilita el control de la alacena de forma premium y moderna, sino que además contribuye de forma tangible al bienestar económico del hogar y a un consumo socialmente responsable.